CMSC131: Encapsulation, Exceptions, Testing, and Debugging

Chapter 7

By Naren Ayinala

May 2026

Contents

1 Why Programs Need More Than Working Code **3**

2 Encapsulation: Why Fields Are Usually Private **3**

2.1 Access Modifiers and Visibility 4

3 Getters and Setters: Reading and Updating State **4**

3.1 Why Not Every Field Needs a Setter 5

3.2 A Getter Can Still Leak Mutable State 5

4 What Is an Exception? **6**

4.1 Compile Errors, Runtime Exceptions, and Logic Errors 6

5 Reading Exception Messages and Stack Traces **6**

6 Throwing Exceptions Deliberately **6**

7 Handling Exceptions with try and catch **7**

7.1 Multiple Catch Blocks and Order 7

8 finally and Resource Cleanup **8**

9 Checked and Unchecked Exceptions **8**

10 Designing a Method's Failure Behavior **9**

11 Why We Test Programs **9**

11.1 A Test Has Three Parts 9

12 Unit Testing with JUnit **10**

12.1 Useful JUnit Assertions 10

13 Testing Exceptions **11**

14 Debugging: Finding the Cause, Not Just the Symptom **11**

14.1 A Loop Bug: Trace Every Iteration 11

14.2 Using a Debugger 12

15 A Complete Example: Account Validation and Tests **12**

16 Putting the Ideas Together **13**

1 Why Programs Need More Than Working Code

A program can compile successfully and still be wrong. For example, a bank account method might allow a negative deposit, a grading method might return the wrong letter for a boundary score, or a method might crash when passed `null`. Writing the happy path is only part of programming. We must also control how objects can be changed, decide what to do when an operation is invalid, and systematically verify behavior.

This chapter follows one `BankAccount` example through four connected ideas: **encapsulation** protects an object's state; **exceptions** report invalid operations; **testing** checks expected behavior; and **debugging** helps find the cause when behavior differs from expectations.

2 Encapsulation: Why Fields Are Usually Private

Consider a bank account with an owner's name and a balance. If the balance is public, any code can assign a negative number without following the account's rules.

```
public class BankAccount {
    public double balance;
}
// Somewhere else:
BankAccount account = new BankAccount();
account.balance = -100000; // No validation!
```

A better design makes fields `private`. The class then controls how other code reads and changes its state. This is called **encapsulation**. The idea is not simply to hide data: it is to ensure that the object remains valid after every permitted operation.

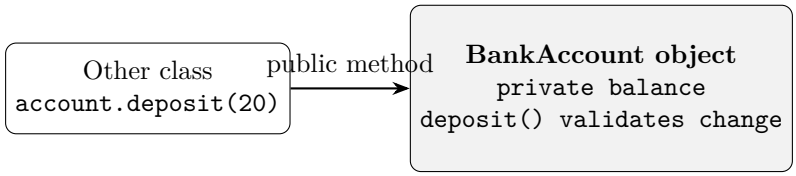
```
public class BankAccount {
    private String owner;
    private double balance;

    public BankAccount(String owner, double balance) {
        if (owner == null || owner.isBlank()) {
            throw new IllegalArgumentException("Owner required");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Negative balance");
        }
        this.owner = owner;
        this.balance = balance;
    }

    public String getOwner() { return owner; }
    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Positive amount required");
        }
        balance += amount;
    }
}
```

The constructor checks the initial state. The `deposit` method checks later changes. There is deliberately no unrestricted `setBalance`: a bank account should change through meaningful operations such as deposits and withdrawals, not arbitrary assignments. In real financial software, use a precise money representation such as integer cents or `BigDecimal`; `double` here keeps the example focused on introductory Java.



2.1 Access Modifiers and Visibility

`public` members are accessible wherever the class itself is accessible. `private` members are accessible only within their declaring class (including its methods and constructors). With no access modifier, a member has *package-private* visibility. `protected` also permits access from certain subclasses and code in the same package; its full rules are best studied with inheritance.

Modifier	Introductory meaning
`public`	Exposed as part of the class's usable interface
`private`	Accessible only inside the declaring class
no modifier	Accessible from other classes in the same package
`protected`	Package access plus special subclass access

3 Getters and Setters: Reading and Updating State

A **getter** returns a field's value. A **setter** accepts a new value and may validate it before storing it. Common Java naming conventions use `getX()` and `setX(...)`; boolean properties often use `isX()`.

```
public class Car {
    private String model;
    private int speed;

    public Car(String model) {
        this.model = model;
        this.speed = 0;
    }

    public String getModel() { return model; }
    public int getSpeed() { return speed; }

    public void setSpeed(int newSpeed) {
        if (newSpeed < 0 || newSpeed > 180) {
            throw new IllegalArgumentException("Invalid speed");
        }
        this.speed = newSpeed;
    }
}
```

The setter parameter `newSpeed` is a local variable. `this.speed` refers to the field of the object on which the method was called. Validation happens *before* assignment, so a rejected update leaves the old speed unchanged.

```
Car car = new Car("Civic");
car.setSpeed(55);
System.out.println(car.getSpeed()); // 55
// car.speed = -5; // Compile error: speed is private
```

3.1 Why Not Every Field Needs a Setter

Some values should not change after construction. For example, a student ID might have a getter but no setter. A bank balance should change only through deposits and withdrawals. If a setter would let callers bypass the class's rules, do not provide one.

```
public class Student {
    private final int id;
    private String email;

    public Student(int id, String email) {
        this.id = id;
        setEmail(email); // Reuse validation
    }
    public int getId() { return id; }
    public String getEmail() { return email; }
    public void setEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.email = email;
    }
}
```

The `final` field `id` can be assigned during construction but cannot later be reassigned. A getter does not make the field writable.

3.2 A Getter Can Still Leak Mutable State

A private reference field is not automatically safe if its getter returns a reference to a mutable object. The caller might mutate that object directly.

```
import java.util.ArrayList;
public class Team {
    private ArrayList<String> names = new ArrayList<>();
    public void addName(String name) { names.add(name); }
    public ArrayList<String> getNames() {
        return new ArrayList<>(names); // Defensive copy
    }
}
```

Returning a new list prevents callers from modifying the team's original list through the getter. A copy of the list is not automatically a deep copy of mutable objects stored inside it; here, the elements are immutable Strings.

4 What Is an Exception?

An **exception** is an object representing an abnormal condition during execution. It can interrupt the normal flow of statements and transfer control to a matching exception handler. Some exceptions are thrown automatically by Java, and others are deliberately thrown by programmers.

```
int[] values = {10, 20, 30};
System.out.println(values[3]); // ArrayIndexOutOfBoundsException
System.out.println("This line is not reached");
```

The array has valid indices 0, 1, and 2. Index 3 is invalid, so Java throws an exception before the next print statement executes. If no handler catches it, the current thread terminates and Java usually prints an error message and stack trace.

4.1 Compile Errors, Runtime Exceptions, and Logic Errors

These are different kinds of problems, and they require different approaches.

Kind	Example	What happens
Compile-time error	`int x = "hi";`	Code fails to compile
Runtime exception	`10 / 0` using integers	Program compiles, then throws
Logic error	Using `>=` instead of `>`	Program runs but result may be wrong

5 Reading Exception Messages and Stack Traces

Consider:

```
public class Demo {
    static void show(int[] a) {
        System.out.println(a[5]);
    }
    public static void main(String[] args) {
        int[] nums = {1, 2};
        show(nums);
    }
}
```

The stack trace will identify an `ArrayIndexOutOfBoundsException` and include the location of the failing array access in `show`, followed by the call in `main`. Line numbers depend on where the source is saved. Read the exception type first, then locate the first relevant line of your own code. Later stack frames show how execution reached that line.

6 Throwing Exceptions Deliberately

We can reject invalid arguments using the `throw` statement. `throw` creates or supplies an exception object and stops normal execution of the current statement sequence unless a matching handler catches it.

```
public static int divide(int numerator, int denominator) {
    if (denominator == 0) {
        throw new IllegalArgumentException("Cannot divide by zero");
    }
    return numerator / denominator;
}
```

The keyword `throw` is different from `throws`. `throw` actually raises an exception; `throws` appears in a method declaration to describe exceptions that the method may pass to its caller.

7 Handling Exceptions with try and catch

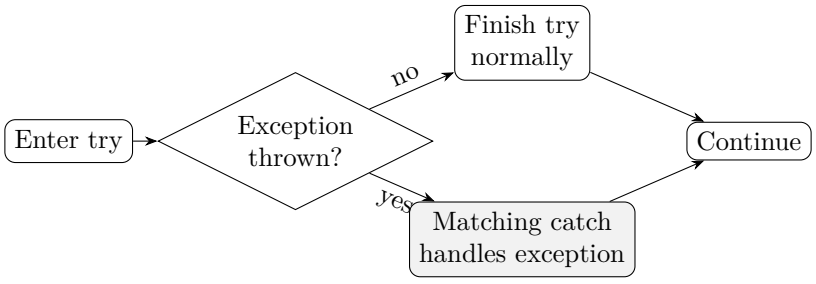
A `try` block contains code that might throw. A `catch` block handles a matching exception. If an exception is thrown, the remaining statements in the `try` block are skipped, and control moves to the appropriate handler.

```
try {
    int answer = 10 / 0;
    System.out.println("Inside try after division");
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
System.out.println("Program continues");
```

Output:

```
Cannot divide by zero
Program continues
```

The skipped print statement inside `try` is important. Java does not resume from the failing statement after the handler completes; it continues after the entire `try/catch` structure (assuming the handler completes normally).



7.1 Multiple Catch Blocks and Order

A handler for a specific exception type must come before a handler for a broader superclass type. Otherwise, the specific handler would be unreachable.

```
try {
    String value = "abc";
    int number = Integer.parseInt(value);
    System.out.println(number);
```

```
} catch (NumberFormatException e) {
    System.out.println("Not a valid integer");
} catch (IllegalArgumentException e) {
    System.out.println("Another invalid argument");
}
```

`NumberFormatException` is a subclass of `IllegalArgumentException`. Reversing these handlers would cause a compile error because the first would already catch the narrower exception.

8 finally and Resource Cleanup

A `finally` block runs after the `try` and any matching `catch`, whether the `try` finishes normally or throws. It is commonly used for cleanup. A `finally` block usually runs even when a method returns from inside `try`; however, it is not guaranteed if the JVM terminates abruptly.

```
try {
    System.out.println("A");
} finally {
    System.out.println("B");
}
System.out.println("C");
// A, B, C on separate lines
```

For resources such as files, Java's `try`-with-resources syntax is generally preferable because it automatically closes resources that implement `AutoCloseable`.

```
import java.util.Scanner;
try (Scanner input = new Scanner("10 20")) {
    System.out.println(input.nextInt()); // 10
}
```

9 Checked and Unchecked Exceptions

Java divides exceptions into **checked** and **unchecked** categories. Checked exceptions generally must be caught or declared using `throws`. Unchecked exceptions are `RuntimeException` and its subclasses; the compiler does not require callers to catch or declare them. `Error` subclasses are also unchecked, but typically represent serious conditions not meant for routine recovery.

```
import java.io.IOException;
public static void readData() throws IOException {
    throw new IOException("Example read failure");
}
```

The `throws IOException` declaration warns callers that they must handle or propagate this checked exception. By contrast, a method throwing `IllegalArgumentException` does not need a `throws` declaration.

Exception	Checked?	Typical cause
`IOException`	Yes	Input/output operation fails
`NumberFormatException`	No	Text cannot be parsed as a number
`NullPointerException`	No	Using `null` as an object reference
`IllegalArgumentException`	No	Method receives an invalid argument
`ArithmeticException`	No	Invalid integer arithmetic, e.g. division by zero
`IndexOutOfBoundsException`	No	Index outside valid range

10 Designing a Method's Failure Behavior

Suppose a withdrawal exceeds the balance. We must decide what the method should do: return a status, return a boolean, or throw an exception. These are different API designs. For invalid arguments, an exception is often appropriate. For an expected business outcome such as insufficient funds, a boolean result might be more convenient in some designs.

```
public boolean withdraw(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Positive amount required");
    }
    if (amount > balance) {
        return false; // Expected unsuccessful withdrawal
    }
    balance -= amount;
    return true;
}
```

This method distinguishes invalid input from an expected refusal. It also preserves the invariant that the balance never becomes negative. Avoid catching every exception and silently ignoring it: doing so can hide programming errors.

11 Why We Test Programs

Testing is the process of checking actual program behavior against a specified expected result. A test is useful only when it has a clear expectation. Printing output and glancing at it can help while learning, but automated assertions make failures easier to detect and repeat.

11.1 A Test Has Three Parts

A common structure is **Arrange, Act, Assert**. First prepare the objects and values. Then perform the operation. Finally check the expected result.

```
// Arrange
Car car = new Car("Civic");
// Act
car.setSpeed(40);
// Assert (manual version)
```

```
System.out.println(car.getSpeed() == 40); // true
```

In an automated test, the last line becomes an assertion so that a mismatch causes a reported test failure instead of merely printing `false`.

12 Unit Testing with JUnit

JUnit is a Java testing framework. The following examples use **JUnit 4**, where `@Test` marks a test method and assertions come from `org.junit.Assert`. JUnit 5 uses different imports and some different conventions.

```
import static org.junit.Assert.*;
import org.junit.Test;

public class CarTest {
    @Test
    public void speedStartsAtZero() {
        Car car = new Car("Civic");
        assertEquals(0, car.getSpeed());
    }

    @Test
    public void setterUpdatesSpeed() {
        Car car = new Car("Civic");
        car.setSpeed(45);
        assertEquals(45, car.getSpeed());
    }
}
```

`assertEquals(expected, actual)` compares the expected and actual values. The `@Test` annotation tells JUnit to run each annotated method as a test. Test methods should be independent: do not assume one test runs before another.

12.1 Useful JUnit Assertions

Assertion	Meaning
`assertEquals(a,b)`	Expected and actual values compare equal
`assertTrue(condition)`	Condition must be true
`assertFalse(condition)`	Condition must be false
`assertNull(object)`	Reference must be null
`assertNotNull(object)`	Reference must not be null
`assertSame(a,b)`	References must point to the same object
`assertNotSame(a,b)`	References must point to different objects
`fail("reason")`	Explicitly fail the test

`assertEquals` and `assertSame` are not interchangeable. For two distinct Strings containing the same characters, content equality succeeds but identity equality fails.

```
String a = new String("cat");
String b = new String("cat");
assertEquals(a, b); // Passes: equal String contents
```

```
assertNotSame(a, b); // Passes: different objects
```

For floating-point results in JUnit 4, use an overload with a tolerance, such as `assertEquals(3.5, result, 0.00001)`.

13 Testing Exceptions

A method's contract can require it to reject invalid input. A good test verifies that invalid input triggers the intended exception rather than silently changing state.

```
import org.junit.Test;
public class CarValidationTest {
    @Test(expected = IllegalArgumentException.class)
    public void negativeSpeedIsRejected() {
        Car car = new Car("Civic");
        car.setSpeed(-1);
    }
}
```

This JUnit 4 form passes if the expected exception is thrown during the test. To also check that the object remains unchanged, a `try`/`catch` test can make the state check explicit:

```
import static org.junit.Assert.*;
import org.junit.Test;
public class CarStateTest {
    @Test
    public void invalidSpeedPreservesOldValue() {
        Car car = new Car("Civic");
        car.setSpeed(30);
        try {
            car.setSpeed(-5);
            fail("Expected IllegalArgumentException");
        } catch (IllegalArgumentException expected) {
            assertEquals(30, car.getSpeed());
        }
    }
}
```

14 Debugging: Finding the Cause, Not Just the Symptom

When a test fails, begin with the difference between *expected* and *actual* behavior. Reproduce the failure with the smallest useful input. Locate the first place where the program's state differs from what you expected. Then correct the underlying cause and rerun the test.

14.1 A Loop Bug: Trace Every Iteration

```
public static int sumTo(int n) {
    int sum = 0;
    for (int i = 1; i < n; i++) { // BUG: misses n
        sum += i;
    }
```

```
    return sum;
}
```

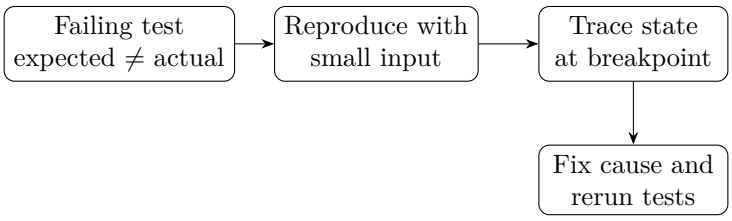
For `sumTo(3)`, the intended sum is $1 + 2 + 3 = 6$. The buggy code produces 3 because it runs for `i = 1` and `i = 2`, then stops when `i = 3`. Change `i < n` to `i <= n` after confirming that the intended range includes `n`.

`i`	`i < 3`?	`sum` before	`sum` after
1	true	0	1
2	true	1	3
3	false	3	no iteration

14.2 Using a Debugger

A debugger lets you pause execution at a **breakpoint**, inspect variable values, and execute one step at a time. **Step over** executes the current line without entering a called method; **step into** follows the call into that method; **step out** finishes the current method and returns to its caller. The exact button names vary by IDE.

Imagine pausing at `car.setSpeed(55)`. Step into to see the parameter `newSpeed`, watch the validation condition, and observe `this.speed` changing. This is more reliable than guessing which line failed from the final output alone.



15 A Complete Example: Account Validation and Tests

This example brings encapsulation, getters, exceptions, and testing together. The account has a private balance, a read-only getter, and methods that preserve the nonnegative-balance rule.

```
public class SimpleAccount {
    private int cents;

    public SimpleAccount(int startingCents) {
        if (startingCents < 0) {
            throw new IllegalArgumentException("Negative balance");
        }
        cents = startingCents;
    }

    public int getCents() { return cents; }

    public void deposit(int amount) {
        if (amount <= 0 || amount > Integer.MAX_VALUE - cents) {
            throw new IllegalArgumentException("Invalid deposit");
```

```
        }
        cents += amount;
    }

    public boolean withdraw(int amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid withdrawal");
        }
        if (amount > cents) return false;
        cents -= amount;
        return true;
    }
}
```

```
import static org.junit.Assert.*;
import org.junit.Test;
public class SimpleAccountTest {
    @Test
    public void depositAndWithdraw() {
        SimpleAccount account = new SimpleAccount(1000);
        account.deposit(250);
        assertEquals(1250, account.getCents());
        assertTrue(account.withdraw(200));
        assertEquals(1050, account.getCents());
    }

    @Test
    public void insufficientFundsDoesNotChangeBalance() {
        SimpleAccount account = new SimpleAccount(100);
        assertFalse(account.withdraw(101));
        assertEquals(100, account.getCents());
    }

    @Test(expected = IllegalArgumentException.class)
    public void negativeStartingBalanceRejected() {
        new SimpleAccount(-1);
    }
}
```

Notice how the tests use only the *public interface*. They cannot directly change the private `cents` field. This is the advantage of a well-encapsulated design: the tests exercise the same rules that real callers must follow.

16 Putting the Ideas Together

When designing a new class, begin by deciding what a valid object looks like. Make fields private unless there is a specific reason not to. Use constructors to establish valid initial state, getters to expose appropriate information, and setters or behavior-specific methods to control updates. Choose deliberate failure behavior for invalid inputs. Write tests for normal behavior, boundaries, and rejected inputs. When something fails, trace execution and inspect state until you find the first incorrect step.

A correct program is not merely one that produces the expected answer for one example. It is one whose design makes invalid states difficult to create, whose failures are understandable, and whose behavior can be checked repeatedly.